AMAZON REVIEW SENTIMENT ANALYSIS

Machine Learning Model Final Report

Abstract

This project leverages machine learning to analyze Amazon reviews, predicting sentiment (positive or negative) to provide actionable insights for businesses. Utilizing a dataset of 50,000 reviews from Kaggle, two models were developed: a Logistic Regression model (baseline) and an advanced Multi-Layer Perceptron (MLP) with LSTM. The Logistic Regression used TF-IDF features for simplicity, while the MLP combined text-based and numerical features, achieving superior accuracy of 93%. A Flask-based web application was built for user interaction, enabling real-time sentiment predictions. Future improvements include aspect-specific sentiment analysis and advanced Transformer-based models.

Shiwakoti Pooja (912581587), Ghimire Kamal (912589739)

Pooja.shiwakoti@uky.edu, kamal.ghimire@uky.edu

Table of Contents

Team Contributions

1. Team Member 1: **Pooja Shiwakoti (**912581587**)**
    - Data preprocessing and feature engineering.
    - Implementation and training of the Logistic Regression model.
2. Team Member 2: **Kamal Ghimire (**912589739**)**
    - Design and training of the MLP model.
    - Integration of numerical features and deployment using Flask.

Amazon Review Sentiment Analysis: Final Report

Introduction

The primary objective of this project was to analyze Amazon reviews and predict the sentiment (positive or negative) using machine learning models. This project aims to provide businesses with actionable insights into customer opinions, helping them improve their products and marketing strategies. The project included two key components: a Logistic Regression model (baseline) and a Multi-Layer Perceptron (MLP) model with LSTM for enhanced performance. This topic is relevant due to the growing importance of customer feedback analysis in e-commerce. Businesses rely heavily on customer reviews to refine their offerings, and tools like this can automate and optimize that process.

Challenges and Solutions:

1. **Dataset Imbalance**: Many reviews had overwhelmingly positive sentiments. To address this, we sampled a balanced subset of 50,000 reviews from the dataset.
2. **Complex Relationships in Text Data**: Logistic Regression struggled with nuanced language. We introduced the MLP model with LSTM, which better captures these complexities.
3. **Multi-Input Feature Integration**: Incorporating numerical features like helpfulness scores was challenging. We resolved this by designing a concatenation layer in the MLP model.

Datasets

Dataset Overview:

The dataset used was ‘Reviews.csv’, a publicly available dataset of Amazon product reviews sourced from **Kaggle.com**. A subset of 50,000 samples was extracted for faster processing and to balance sentiment distribution.

Feature	Description
Text	The review content, used for sentiment analysis.
Score	Product rating (converted to binary: positive if ≥3, negative otherwise).
HelpfulnessNumerator	Number of users who found the review helpful.
HelpfulnessDenominator	Total number of users who rated the review’s helpfulness.

Sample Dataset:

Id	Text	Score	HelpfulnessNumerator	HelpfulnessDenominator
1	I have bought several of the Vitality canned dog food products and found them all good.	1	10	12
2	Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized.	0	5	20

Aspect-Specific Labels (Optional Extension):

The proposal included aspect-specific sentiment analysis for attributes like "value for money" and "durability." However, this was not implemented in the current project. Future improvements should focus on deriving these labels using keywords and machine learning techniques.

Evaluation Metrics

1. **Accuracy**: Measures the percentage of correct predictions.
2. **Precision, Recall, and F1-Score**: Provide a more detailed view of model performance, particularly for imbalanced datasets.
3. **Confusion Matrix**: Offers insights into false positives and false negatives.

These metrics were chosen to comprehensively evaluate both the baseline and advanced models, ensuring fairness in comparison.

Baseline Model

The baseline model used Logistic Regression with TF-IDF vectorized features. It provides a simple and interpretable approach for sentiment classification. TF-IDF captures the importance of words in the review text, helping the model focus on relevant features.

Details of Our Method

Approach:

1. **Preprocessing**:
    - Converted text to lowercase and removed punctuation.
    - Tokenized and stemmed words using NLTK.
    - Applied TF-IDF vectorization for Logistic Regression.
    - Tokenized and padded sequences for MLP.

2. **Logistic Regression**:
    - Used TF-IDF features.
    - Hyperparameters: ‘max_iter=300’, solver=’saga’.

3. MLP with LSTM:
    - Used a combined input of padded sequences and numerical features.
    - Architecture:
        - **Embedding Layer**: Converts text to dense vectors.
        - **LSTM Layer**: Captures sequential relationships in text.

    - **Concatenation Layer**: Integrates LSTM output with numerical features.
    - **Dense Layer**: Final classification.
- Hyperparameters:
- Embedding dimension: 128
- LSTM units: 64
- Dropout rate: 0.3
- Batch size: 64
- Epochs: 3

Core Implementation:

```
# Define MLP Model
input_text = Input(shape=(150,), name="text_input")
input_helpfulness = Input(shape=(2,), name="helpfulness_input")

embedding_layer = Embedding(input_dim=20000, output_dim=128, input_length=150)(input_text)
lstm_out = LSTM(64, dropout=0.2)(embedding_layer)
concat_layer = Concatenate()([lstm_out, input_helpfulness])
dropout_layer = Dropout(0.3)(concat_layer)
output_layer = Dense(1, activation='sigmoid')(dropout_layer)

mlp_model = Model(inputs=[input_text, input_helpfulness], outputs=output_layer)
mlp_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Results

Logistic Regression Performance:

Metric	Class 0 (negative)	Class 1 (Positive)
Precision	0.79	0.91
Recall	0.41	0.98
F1 – Score	0.54	0.95
Support	1405	8595

Accuracy: 90%

Macro Average: Precision = 0.85, Recall = 0.70, F1-Score = 0.74

Weighted Average: Precision = 0.89, Recall = 0.90, F1-Score = 0.89

Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	576	829
Actual Positive	152	8443

Performance Comparison:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	90%	0.91	0.98	0.95
MLP with LSTM	93%	0.94	0.97	0.96

Analysis:

- **MLP Outperformed Logistic Regression**: The LSTM layer captured sequential dependencies in text, while the concatenation layer effectively integrated numerical features.

- **Challenges**:
    - Logistic Regression struggled with nuanced reviews due to its reliance on TF-IDF features.
    - MLP required more computational resources.

Potential Improvements:

- **Aspect-Specific Sentiment Analysis**: Implement aspect-specific sentiment classification as proposed, focusing on attributes like "value for money" and "durability."
- Extend MLP to include pre-trained embeddings like GloVe.
- Fine-tune hyperparameters for better performance.

Future Work and Lessons Learned

Future Extensions:

1. **Aspect-Specific Analysis**: Implement aspect-specific sentiment classification to align with the original proposal.
2. **Advanced Models**: Explore Transformer-based models like BERT for state-of-the-art results.

Lessons Learned:

1. Importance of feature engineering and preprocessing.
2. The trade-off between model complexity and interpretability.
3. Efficient handling of imbalanced datasets.

User Interface

The project includes a Flask-based web application for testing the sentiment analysis models.

Key Features:

1. **Input Interface**: Users can input review text directly in a text box.
2. **Backend Processing**: Flask communicates with pre-trained Logistic Regression and MLP models to generate predictions.
3. **Output Display**: The sentiment (positive or negative) is displayed on the web page.

Steps to Run:

1. Navigate README.md file and follow the instruction from top to bottom
2. Launch the Flask application using:

```
python app/app.py
```

2. Access the app in a web browser at http://127.0.0.1:5000.
3. Enter a review in the text box and click the "Predict Sentiment" button.

Conclusion

The project successfully demonstrated sentiment analysis using both a baseline Logistic Regression model and an advanced MLP model with LSTM. The addition of numerical features like helpfulness scores significantly improved the MLP model's performance. The project lays the groundwork for future exploration into aspect-specific sentiment analysis and advanced deep learning techniques.